

Introduction to Use Cases in Software Engineering

1 Introduction

In software engineering, a **Use Case** is a fundamental tool used to describe how a user interacts with a system to achieve a particular goal. It provides a user-focused perspective of system behavior and acts as a bridge between what users expect and what developers implement.

Formal Definition

A **Use Case** is a textual or diagrammatic description of a system's behavior in response to requests from external users (actors) that achieve a specific goal.

Use cases are essential for translating high-level user needs into actionable functional requirements that guide design, testing, and validation.

2 Purpose of a Use Case

The main purpose of a use case is to capture and communicate the **functional requirements** of a system in a user-oriented manner.

- Defines what the system should do, not how it should be implemented.
- Identifies primary users (**actors**) and their interactions with the system.
- Serves as a shared understanding medium among **stakeholders**, **analysts**, and **developers**.

Key Idea

A well-written use case ensures that software solutions align with real-world user goals rather than purely technical specifications.

3 Context in Software Development

Use cases are typically developed during the **Requirements Analysis Phase** of the **Software Development Life Cycle (SDLC)**. They serve as the foundation for subsequent processes such as design, testing, and validation.

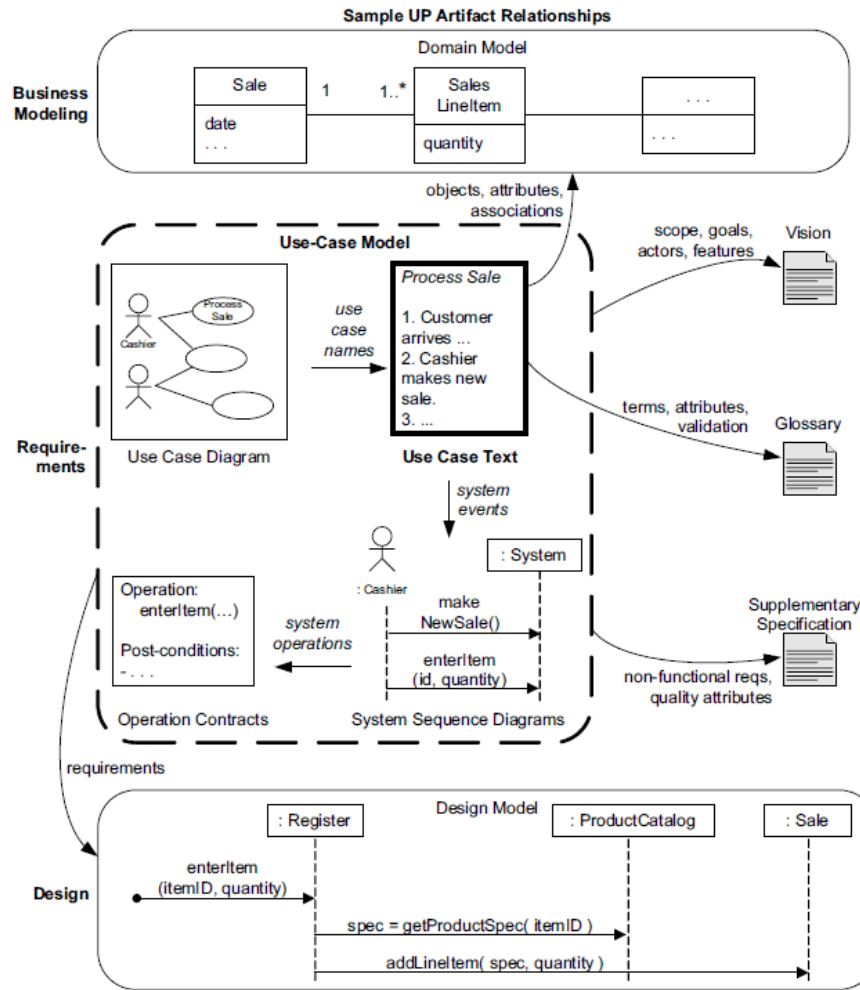


Figure: Use cases bridge the gap between user requirements and technical design.

4 Key Characteristics of Use Cases

A high-quality use case exhibits the following characteristics:

- 1. User-Centric:** Focuses on what the user wants to achieve, not internal implementation details.
- 2. Goal-Oriented:** Each use case represents a meaningful goal that provides value to an actor.
- 3. Scenario-Based:** Describes the sequence of interactions between the user and system.
- 4. System Boundary Defined:** Clearly identifies which functions belong inside or outside the system.

5. **Understandable:** Written in natural, non-technical language for easy stakeholder comprehension.

5 Importance of Use Cases

Use cases ensure that the system meets user expectations and business objectives. They serve as a foundation for planning, design, testing, and communication across the development team.

Why Use Cases Are Important

Use cases act as a bridge between the user's perspective and system functionality, providing a structured representation of expected behaviors and goals.

Key benefits include:

- Enhance communication between clients, users, and developers.
- Simplify functional requirement documentation.
- Detect missing or conflicting requirements early.
- Serve as the basis for **test cases**, **user stories**, and **system design**.

6 Example: Online Banking Login

Consider an **Online Banking System** where a customer logs into their account.

Use Case Example – Login to Account

Actor: Bank Customer

Goal: Access online banking account.

Precondition: User must have a valid registered account.

Main Flow:

1. The customer navigates to the login page.
2. Enters a valid username and password.
3. The system verifies credentials.
4. Upon success, the system redirects to the dashboard.

Alternative Flow:

1. If credentials are invalid, the system displays an error message and prompts re-entry.

Postcondition: User is authenticated and granted access.

This example demonstrates how a use case captures normal and exceptional paths of user-system interaction.

7 Relation to Other Modeling Tools

Use cases complement other **UML (Unified Modeling Language)** diagrams, forming a complete modeling ecosystem.

- **Activity Diagrams** – Visualize workflow logic of a use case.
- **Class Diagrams** – Show data structures and relationships supporting the use case.
- **Sequence Diagrams** – Illustrate message flow among components during execution.

8 Summary

In summary, a **Use Case** is a narrative representation of user-system interaction that captures goals, conditions, and outcomes. It provides a human-centric approach to requirement analysis and ensures that system behavior aligns with stakeholder expectations.